



Universidad del Valle de Guatemala
Facultad de Ingeniería
CC3092 — Deep Learning y Sistemas Inteligentes

Proyecto #2

Competencia de Agentes en Space Invaders (Deep Q-Learning)

Nicolás Concuá
Guatemala, octubre de 2026

Enlace al repositorio: https://github.com/nicoCT04/Deep_Q_Learning

1. Definición del problema y análisis del entorno

El objetivo es entrenar un agente de Aprendizaje por Refuerzo capaz de jugar *Space Invaders* sobre el entorno **ALE/SpacInvaders-v5** de Gymnasium, documentando todo el proceso de desarrollo. Antes de entrenar se analiza el entorno, de forma análoga al EDA de un problema supervisado [1, 2].

1.1 El entorno

- **Objetivo:** destruir oleadas de invasores maximizando el puntaje, esquivando sus disparos y evitando que lleguen al fondo.
- **Recompensa:** puntos por destruir invasores; los de filas superiores valen más, por lo que la recompensa por paso toma valores discretos (múltiplos de 5).
- **Vidas y fin de episodio:** el agente inicia con 3 vidas. **terminated** ocurre al agotarlas; **truncated** al alcanzar el tope de 108000 frames.
- **Configuración de la versión -v5:** frameskip = 4, repeat_action_probability = 0.25 (sticky actions) y espacio de acción mínimo, siguiendo el protocolo de evaluación estándar [3].

1.2 Espacio de observación y preprocesamiento

La observación cruda es una imagen RGB de 210×160×3 (uint8). Se aplica el preprocesamiento estándar de DQN [1]: escala de grises, redimensionado a 84×84, apilado de 4 frames consecutivos (para capturar movimiento) y normalización de píxeles.



Figura 1. Observación RGB cruda (210×160×3) vs. preprocesada (gris, 84×84).

1.3 Espacio de acción

El espacio de acción es **Discrete(6)**: NOOP, FIRE, RIGHT, LEFT, RIGHTFIRE y LEFTFIRE. Se usa este subconjunto mínimo en lugar de las 18 acciones genéricas de Atari, lo que reduce la salida de la red y acota la exploración a acciones con sentido, acelerando el aprendizaje.

1.4 Análisis de la señal de recompensa

La recompensa es **dispersa** (solo ~2.5% de los pasos recibe recompensa no nula) y de magnitud heterogénea (múltiplos de 5). Por ello se aplica *reward clipping* al signo $\{-1, 0, +1\}$ [1], que estabiliza el aprendizaje de los valores Q. No se usó *reward shaping* adicional. El puntaje real (sin recortar) se mide aparte en la evaluación.

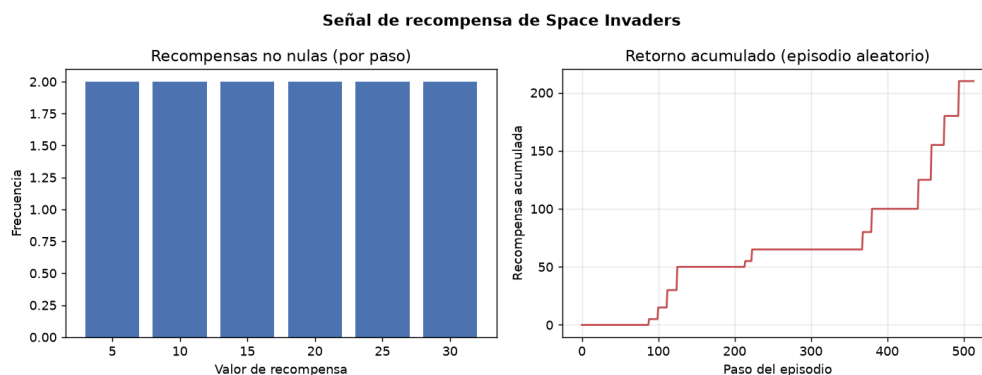


Figura 2. Distribución de recompensas no nulas y retorno acumulado de un episodio aleatorio.

2. Metodología de desarrollo

2.1 Algoritmo

Se eligió **DQN (Deep Q-Network)** [1] con Stable-Baselines3 [4]. Space Invaders tiene un espacio de acción discreto (6 acciones), escenario natural de DQN, que aprende una función de valor $Q(s,a)$ y actúa por argmax . DQN incorpora *replay buffer* y *red objetivo*, las dos ideas que estabilizan el aprendizaje con redes profundas. El enunciado permite usar librería documentando las decisiones.

2.2 Arquitectura de la red

CNN de la publicación original de DQN (NatureCNN, la CnnPolicy por defecto de SB3), con entrada $4 \times 84 \times 84$:

- **Conv1**: 32 filtros 8×8 , stride 4, ReLU.
- **Conv2**: 64 filtros 4×4 , stride 2, ReLU.
- **Conv3**: 64 filtros 3×3 , stride 1, ReLU.
- **Densa**: 512 unidades ReLU, y salida lineal de 6 valores Q (uno por acción).

2.3 Exploración vs. explotación

Estrategia ϵ -greedy con ϵ decreciente de 1.0 a 0.01 durante el 10% inicial del entrenamiento. En la evaluación se usa $\epsilon = 0$ (política greedy pura), como exige la competencia.

2.4 Hiperparámetros de entrenamiento

- Optimizador Adam, pérdida Huber, tasa de aprendizaje $1e-4$, factor de descuento $\gamma = 0.99$.
- Replay buffer de 80-100k transiciones, `learning_starts` = 100 000, `batch` = 32.
- `train_freq` = 4, actualización de la red objetivo cada 1 000 pasos, `reward clipping` activo.
- Total del modelo final: 8 000 000 de pasos, con checkpoints cada 500 000.

3. Resultados de iteraciones

Se compararon tres iteraciones. La primera se entrenó en Google Colab (T4); las dos siguientes, más largas (8 M pasos), se entrenaron **localmente en un MacBook Pro M4 Pro con MPS**, corriéndolas en paralelo (se midió $\sim 1.74x$ de throughput total, pues en DQN de un solo entorno la GPU queda subutilizada esperando a la CPU).

Iteración	Configuración	Pasos	Entorno	Puntaje greedy (prom / máx)
It.1	DQN base (LR $1e-4$)	2 M	Colab (T4)	625 / 1255
It.2 (final)	DQN base (LR $1e-4$)	8 M	Local (MPS)	1022 / 1815
It.3	DQN (LR $2.5e-4$)	8 M	Local (MPS)	648 / 1175

Hallazgos: (a) **más pasos = mucho mejor** — pasar de 2 M a 8 M casi duplicó el promedio ($625 \rightarrow 1022$) y subió el máximo a 1815; (b) **subir la tasa de aprendizaje a $2.5e-4$ (It.3) empeoró el resultado**, no lo mejoró. Correr It.2 e It.3 en paralelo permitió compararlas y quedarse con la ganadora.

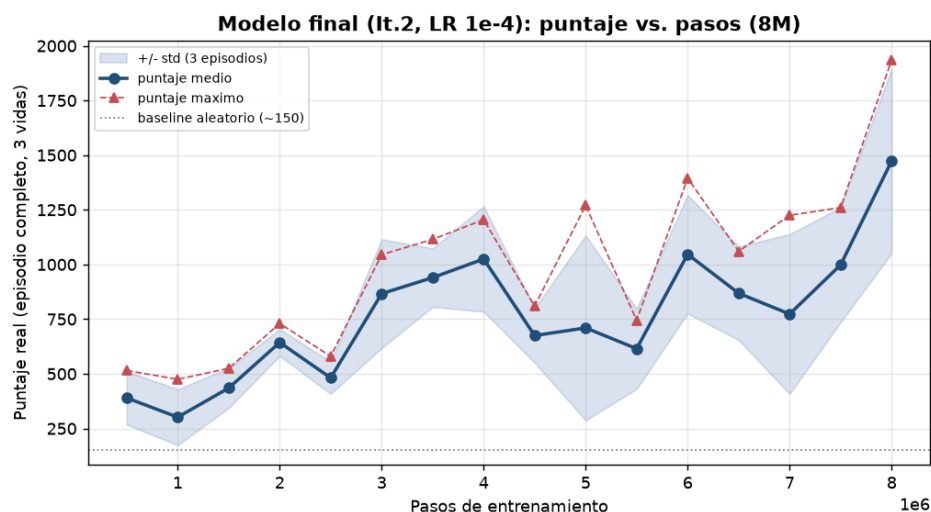


Figura 3. Modelo final (It.2, 8M): puntaje real vs. pasos de entrenamiento (evaluando sus checkpoints).

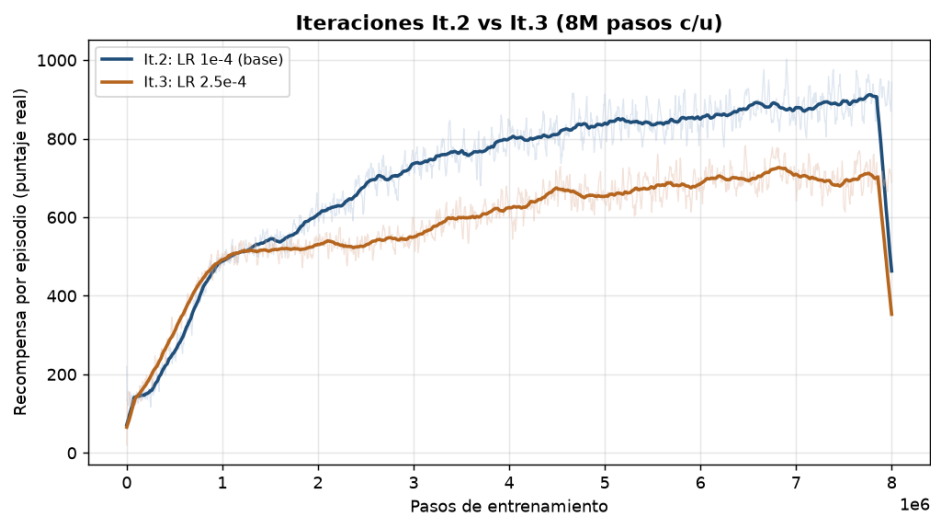


Figura 4. Comparación de iteraciones It.2 (LR $1e-4$) vs It.3 (LR $2.5e-4$), 8M pasos.

3.1 Problemas encontrados y cómo se abordaron

- **Corte de energía / desconexión de Colab (It.1, ~663 k pasos):** gracias al guardado de checkpoints se reanudó desde el último sin perder progreso. Motivó llevar el entrenamiento largo a la máquina local, más estable.
- **Tasa de aprendizaje alta (It.3) empeoró el desempeño:** evidencia directa del impacto de este hiperparámetro.
- **Pérdida creciente pero estable:** los valores Q crecen con el retorno esperado; la pérdida sube pero no diverge.
- **Alta varianza entre episodios:** por los sticky actions (0.25); por eso la competencia toma el máximo de varios episodios.

4. Discusión de resultados

- **Mayor impacto:** la cantidad de pasos. El puntaje real sube marcadamente hacia los 8 M pasos, con promedio ~1022 y máximos por encima de 1800.
- **Comportamiento cualitativo (video):** el agente dispara casi continuamente mientras se desplaza para alinear tiros y esquivar proyectiles. Falla cuando varios disparos convergen sobre su posición o cuando se arrincona, y no prioriza la nave nodriza.

- **Cómputo:** el entrenamiento largo se hizo local en un M4 Pro con MPS, más rápido que la T4 de Colab para este DQN (~285 vs ~170 pasos/s por proceso) y sin desconexiones.
- **Trade-off exploración/explotación:** con ϵ alto el puntaje es bajo; al decaer a 0.01 el desempeño mejora, y la política greedy ($\epsilon = 0$) lo maximiza.

5. Conclusiones

- El agente DQN final alcanza 1815 (máximo) / 1022 (promedio) en evaluación greedy (20 episodios), muy por encima del baseline aleatorio (~150) y de la iteración inicial de 2 M (1255 / 625).
- El preprocesamiento estándar de Atari y el guardado de checkpoints fueron decisivos; el segundo permitió reanudar tras la desconexión de Colab.
- Aprendizaje metodológico: más pasos de entrenamiento fue la mejora más efectiva; subir la tasa de aprendizaje puede perjudicar; y separar el entorno de entrenamiento del de evaluación es clave para medir el puntaje real.
- Trabajo futuro: Double DQN y Dueling DQN, prioritized experience replay, aún más pasos, y comparación contra PPO/A2C.

Referencias

- [1] V. Mnih et al. (2015). Human-level control through deep reinforcement learning. Nature, 518, 529-533.
- [2] Farama Foundation (2024). Gymnasium & Arcade Learning Environment (ALE) documentation. <https://gymnasium.farama.org/environments/atari/>
- [3] M. C. Machado et al. (2018). Revisiting the Arcade Learning Environment. JAIR, 61.
- [4] A. Raffin et al. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. JMLR, 22.
- [5] H. van Hasselt, A. Guez, D. Silver (2016). Deep Reinforcement Learning with Double Q-learning. AAAI.